ОПЕРАЦИОННЫЕ СИСТЕМЫ (WINDOWS) • КОНТРОЛЬНЫЙ ПИСЬМЕННЫЙ ОПРОС И ПРАКТИКУМ

Полное пошаговое руководство к выполнению Варианта 22

Ниже приведено полное пошаговое руководство к выполнению **Варианта 22** для всех четырёх разделов письменного опроса в операционной системе **Windows** (фамилия студента — **Sokolov**).

РАЗДЕЛ 1. ОБЩАЯ ТЕОРИЯ

НОМЕР ВОПРОСА: 22 (СООТВЕТСТВУЕТ НОМЕРУ ВАРИАНТА) • ФОРМУЛИРОВКА ВОПРОСА

Вытеснение процесса. Объясните переход из выполняющегося состояния в готовое и отличие этого перехода от перехода в ожидание. Какую роль играет планировщик при последующем выборе процесса? Приведите пример ситуации, в которой происходит вытеснение.

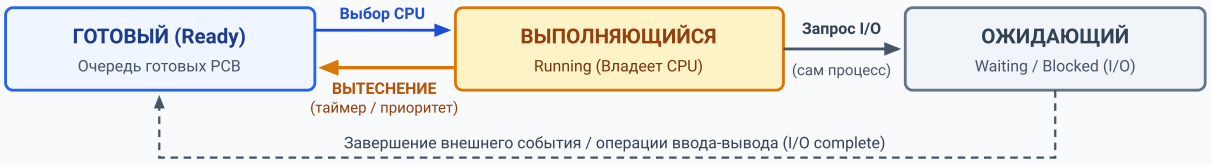


Рис. 1. Сравнение принудительного вытеснения ядром (Running → Ready) и добровольной блокировки (Running → Waiting)

Ответ для отчёта

1. **Понятие вытеснения:**
   Вытеснение (preemption) — это механизм операционной системы, при котором планировщик принудительно забирает процессор у текущего выполняющегося процесса до момента его добровольного освобождения или завершения, временно передавая CPU другому процессу.
2. **Переход «Выполняющийся → Готовый» и его отличие от «Выполняющийся → Ожидающий»:**
   - **Переход «Выполняющийся → Готовый» (Running → Ready):** Процесс полностью работоспособен, не ожидает внешних данных или сигналов, и для продолжения работы ему требуется исключительно процессорное время. Перевод в состояние готовности инициируется ядром ОС (например, по прерыванию аппаратного таймера или по факту появления более приоритетной задачи).
   - **Переход «Выполняющийся → Ожидающий/Блокированный» (Running → Waiting/Blocked):** Инициируется самим процессом при выполнении синхронного системного вызова (чтение с диска, ожидание сетевого пакета, ввод с клавиатуры, ожидание семафора/события). В этом состоянии процессор не может быть выделен процессу, даже если он свободен, вплоть до наступления запрашиваемого внешнего события.
3. **Роль планировщика при последующем выборе процесса:**
   При вытеснении планировщик сохраняет контекст процессора (регистры, указатель стека, программный счётчик) в структуру PCB (Process Control Block) вытесненного процесса и помещает его в очередь готовых процессов. Затем, на основе заложенного алгоритма диспетчеризации (например, Round Robin, Multilevel Feedback Queue или на основе статических/динамических приоритетов), планировщик выбирает из очереди следующий готовый процесс, загружает его контекст и передаёт ему процессорное время.
4. **Пример ситуации вытеснения:**
   - **По кванту времени (Time Slice Expiration):** В системах с алгоритмом Round Robin (карусельное планирование) процессу выделяется фиксированный квант (например, 20 мс). Если за этот квант процесс производит длинный цикл вычислений, аппаратный таймер генерирует прерывание, ОС прерывает выполнение процесса, перемещает его в конец очереди готовых задач и передаёт процессор следующему процессу.
   - **По приоритету:** В интерактивной системе фоновый процесс компиляции вытесняется в очередь готовых, когда пользователь нажимает клавишу в текстовом редакторе, так как интерактивный поток имеет более высокий приоритет.

РАЗДЕЛ 2. ТЕОРЕТИЧЕСКИЕ ВОПРОСЫ ПРАКТИЧЕСКИХ РАБОТ

Согласно правилу формирования номеров для Раздела 2, при варианте 22 берутся **Вопрос 22** (работа с файлами) и **Вопрос 52** (работа с процессами).

> **ВОПРОС 22 (ТЕМА: ФАЙЛЫ И КАТАЛОГИ)**
> **Объясните назначение ключей /S и /Q команды rd в Windows. Сравните выполнение рекурсивного удаления с запросом подтверждения и без него, указав различия в действиях пользователя и рисках.**

Ответ для отчёта (Вопрос 22)

- **Назначение ключей команды `rd` (`rmdir`):**
  - По умолчанию команда **`rd <каталог>`** удаляет **только пустой** каталог.
  - Ключ **`/S`** (Subtree): указывает удалить указанный каталог вместе со всеми вложенными в него подкаталогами и файлами (рекурсивное удаление дерева).
  - Ключ **`/Q`** (Quiet): режим «тишины». Отключает интерактивный запрос подтверждения у пользователя.
- **Сравнение выполнения с подтверждением и без:**
  - **С запросом подтверждения (`rd /S <имя>`):** Система выводит запрос вида **`имя, вы уверены [Y(да)/N(нет)]?`**. Пользователю требуется явно подтвердить операцию нажатием клавиши **`Y`** и **`Enter`**. Это даёт возможность пользователю визуально проверить имя каталога и предотвращает случайное удаление важной директории при опечатке.
  - **Без запроса подтверждения (`rd /S /Q <имя>`):** Удаление выполняется немедленно без вопросов.
- **Риски:** Использование ключа **`/Q`** несет высокий риск безвозвратной потери данных (файлы, удаленные из командной строки, не попадают в «Корзину»). Любая ошибка в пути или случайный лишний пробел (например, **`rd /s /q my folder`** вместо **`rd /s /q "my folder"`**) приведет к удалению каталога **`my`**, если он существует.

> **ВОПРОС 52 (ТЕМА: УПРАВЛЕНИЕ ПРОЦЕССАМИ)**
> **Опишите завершение процессов по имени в Windows или Linux. Приведите пример команды и объясните, как проверить результат, если совпадений нет либо часть выбранных процессов не удалось завершить.**

Ответ для отчёта (Вопрос 52)

- **Команда завершения по имени в Windows:** Используется утилита **`taskkill`** с параметром **`/IM`** (Image Name):
  ```
  taskkill /IM notepad.exe
  ```
  Для принудительного завершения добавляется ключ **`/F`**:
  ```
  taskkill /IM notepad.exe /F
  ```
- **Поведение и проверка при отсутствии совпадений:** Если в системе нет процессов с указанным именем, **`taskkill`** возвращает ненулевой код ошибки (**`%ERRORLEVEL% = 128`** или **`1`**) и выводит текстовое сообщение: **`ОШИБКА: Процесс "notepad.exe" не найден.`** Проверить результат можно через переменную ошибки:
  ```
  echo %ERRORLEVEL%
  ```
  либо повторным фильтром:
  ```
  tasklist /FI "IMAGENAME eq notepad.exe"
  ```
  (вывод: ИНФОРМАЦИЯ: отсутствуют задачи, удовлетворяющие заданным критериям).
- **Поведение при частичном завершении:** Если одни экземпляры процесса закрылись, а другие защищены правами доступа или заблокированы системным вводом-выводом, **`taskkill`** для каждого PID выдаст отдельное уведомление:
  - Для успешных: **`Успешно: процесс "..." с идентификатором ... был завершен.`**
  - Для неуспешных: **`ОШИБКА: Не удается завершить процесс ... Отказано в доступе.`**

  Фактическое состояние проверяется списком **`tasklist /FI "IMAGENAME eq имя.exe"`** — оставшиеся в

РАЗДЕЛ 3. ПРАКТИЧЕСКАЯ ЗАДАЧКА: «РАБОТА С ФАЙЛАМИ» (ВАРИАНТ 22)

ПОСТАНОВКА ИНДИВИДУАЛЬНОГО ЗАДАНИЯ (ВАРИАНТ 22)

1. Создать каталоги **source**, **copies** и **moved**.
2. В **source** создать **copy_a.txt**, **move_b.txt** и **note.log**.
3. Скопировать файлы с началом имени **copy_** в **copies**, а файлы с началом имени **move_** переместить в **moved**. В обеих операциях использовать маски.
4. Переименовать **moved/move_b.txt** в **second.txt**.
5. Объединить **copies/copy_a.txt** и **moved/second.txt** в **result/actions.txt**. Показать различие в сохранности исходников после копирования и перемещения.

А также выполнить все требования обязательной части!

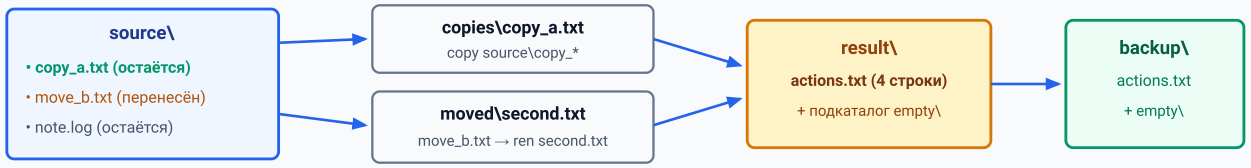


Рис. 2. Схема операций с файлами и каталогами в рабочей области kr1_sokolov_22

Пошаговая инструкция выполнения в Windows (cmd.exe)

Шаг 1. Подготовка рабочей области и демонстрация навигации

Запустите командную строку (**cmd.exe**):

```
chcp 1251
cd %USERPROFILE%
mkdir kr1_sokolov_22
cd kr1_sokolov_22
cd
```

*Вывод команды **cd** подтверждает текущий каталог: **C:\Users\<Имя>\kr1_sokolov_22**. Продемонстрируем переход во вложенный каталог и возврат в родительский:*

```
mkdir nav_demo
cd nav_demo
cd
cd ..
rd nav_demo
```

Шаг 2. Создание обязательной начальной структуры

Создаём каталоги **result/empty**, **evidence**, **discard/inner** и файл **remove.tmp**:

```
mkdir result\empty
mkdir evidence
mkdir discard\inner
(echo Variant 22& echo File remove.tmp) > discard\inner\remove.tmp
```

Шаг 3. Создание каталогов и файлов варианта 22

Создаём каталоги **source**, **copies**, **moved** и файлы с обязательным форматом из двух строк (Variant 22 / File NAME), затем проверяем создание:

```
mkdir source copies moved
(echo Variant 22& echo File copy_a.txt) > source\copy_a.txt
(echo Variant 22& echo File move_b.txt) > source\move_b.txt
(echo Variant 22& echo File note.log) > source\note.log
dir source
```

Шаг 4. Копирование и перемещение по маскам • **Шаг 5. Переименование файла**

Копируем файлы **copy_*** в каталог **copies**, перемещаем файлы **move_*** в каталог **moved** и переименовываем **moved\move_b.txt** в **second.txt**:

```
copy source\copy_* copies\
```

Продолжение Раздела 3: Завершающие шаги работы с файлами (Шаги 7–9)

Шаг 7. Резервное копирование каталога `result` в `backup`

Копируем структуру каталога **`result`** в новый каталог **`backup`** так, чтобы внутри были файл и подкаталог **`empty`** (без лишнего уровня **`result`**), и проверяем структуру:

```
xcopy result backup\ /E /I
dir /s backup
```

Шаг 8. Сохранение справки, удаление `discard` и листинг

Сохраняем справку по средству копирования, проверяем содержимое каталога **`discard`**, удаляем его со всеми вложениями и сохраняем итоговый список файлов и папок рабочей области в **`evidence/listing.txt`**:

```
copy /? > evidence\help_copy.txt
dir /s discard
rd /s /q discard
dir /s > evidence\listing.txt
```

Шаг 9. Итоговая демонстрация результатов (для скриншотов)

```
type result\actions.txt
type backup\actions.txt
type evidence\listing.txt
```

РАЗДЕЛ 4. ПРАКТИЧЕСКАЯ ЗАДАЧКА: «РАБОТА С ПРОЦЕССАМИ» (ВАРИАНТ 22)

> **ПОСТАНОВКА ИНДИВИДУАЛЬНОГО ЗАДАНИЯ (ВАРИАНТ 22)**
>
> **Исходная структура: P → A, B и Q → C, D**; все дети выполняют одну программу (например, **`notepad.exe`**), процесс **U** независим (например, **`mspaint.exe`**).
>
> 1. Получить общий список детей по имени программы.
> 2. Разделить выборку по PPID на два набора.
> 3. Понизить приоритет **D**, затем завершить только детей **Q**, сохранив обоих родителей.
> 4. Доказать, что дети **P** не затронуты. Затем выполнить очистку.
>
> *А также выполнить все требования обязательной части!*

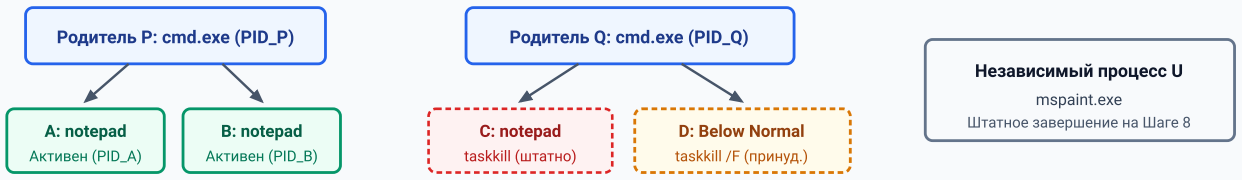


Рис. 3. Целевая иерархия процессов Варианта 22 и селективное управление дочерними ветвями

Пошаговая инструкция выполнения в Windows (Шаги 1–3)

Шаг 1. Создание рабочей области

Откройте основную командную строку (**`cmd.exe`**):

```
chcp 1251
cd %USERPROFILE%
mkdir kr1_process_sokolov_22
cd kr1_process_sokolov_22
```

Шаг 2. Запуск иерархий процессов и независимого процесса U

Чтобы процессы **A**, **B** имели одного родителя **P**, а **C**, **D** — другого родителя **Q**, запускаем два отдельных командных интерпретатора **`cmd.exe`** (они станут процессами **P** и **Q**), каждый из которых породит по два дочерних блокнота **`notepad.exe`**, а также независимый процесс **U** (Paint):

```
start mspaint
start "Parent_P" cmd /k "start notepad & start notepad"
start "Parent_Q" cmd /k "start notepad & start notepad"
```

Продолжение Раздела 4: Выполнение шагов варианта и финальная очистка (Шаги 4–8)

Шаг 4. Шаг 1 варианта — Общий список детей по имени программы
Получаем список всех 4-х дочерних процессов блокнота:

```
tasklist /FI "IMAGENAME eq notepad.exe"
```

*(В таблице отображаются 4 процесса **`notepad.exe`** с разными PID).*

Шаг 5. Шаг 2 варианта — Разделение выборки по PPID на два набора
Выводим процессы с группировкой по **`ParentProcessId`**:

```
powershell "Get-CimInstance Win32_Process -Filter \"Name = 'notepad.exe'\" | Select-Object
ProcessId, ParentProcessId, Name | Sort-Object ParentProcessId | Format-Table -AutoSize"
```

- Два блокнота имеют общий **`ParentProcessId = <PID_P>`**. Обозначим их как **A** (**`PID_A`**) и **B** (**`PID_B`**).
- Другие два блокнота имеют общий **`ParentProcessId = <PID_Q>`**. Обозначим их как **C** (**`PID_C`**) и **D** (**`PID_D`**).

(Зафиксируйте эти конкретные числа PID в тексте отчёта).

Шаг 6. Шаг 3 варианта — Изменение приоритета D и завершение детей Q
1. **Понижаем приоритет процесса D** и проверяем значение приоритета до и после (было Normal/8, стало Below Normal/4):

```
wmic process where ProcessId=<PID_D> CALL setpriority "below normal"
powershell "Get-Process -Id <PID_D> | Select-Object Id, ProcessName, PriorityClass"
```

2. **Завершаем только детей Q (процессы C и D):** завершаем **C** штатно, а **D** — принудительно (с ключом **`/F`**, выполняя требование обязательной части о принудительном завершении):

```
taskkill /PID <PID_C>
taskkill /PID <PID_D> /F
```

Шаг 7. Шаг 4 варианта — Доказательство сохранности детей P и родителей
1. Проверяем, что процессы детей **P** (**A** и **B**) активны (в списке остались ровно два процесса с **`PID_A`** и **`PID_B`**).
2. Проверяем, что оба родителя (**P** и **Q**) продолжают работу:

```
tasklist /FI "IMAGENAME eq notepad.exe"
tasklist /FI "PID eq <PID_P>"
tasklist /FI "PID eq <PID_Q>"
```

Шаг 8. Завершение U и полная очистка (Обязательная часть)
1. Запрашиваем **штатное** завершение независимого процесса **U** (Paint). 2. Завершаем оставшиеся учебные процессы (**A**, **B**, **P**, **Q**). 3. Сохраняем финальную проверку в файл **`own_after.txt`**:

```
taskkill /IM mspaint.exe
taskkill /PID <PID_A> /F
taskkill /PID <PID_B> /F
taskkill /PID <PID_P> /F
taskkill /PID <PID_Q> /F
powershell "Get-CimInstance Win32_Process | Where-Object { $_.Name -match 'notepad|mspaint' }
| Select-Object ProcessId, Name" > own_after.txt
```

*Файл **`own_after.txt`** пуст, что подтверждает корректную очистку системы.*

> **ШПАРГАЛКА ДЛЯ ПОДСТАНОВКИ ВАШИХ РЕАЛЬНЫХ PID ВО ВРЕМЯ ВЫПОЛНЕНИЯ**
> Запишите свои значения из Шага 5 перед вводом команд Шага 6–8: **`PID_P = [____]`** · **`PID_A = [____]`** · **`PID_B = [____]`** · **`PID_Q = [____]`** · **`PID_C = [____]`** · **`PID_D = [____]`**.

Раздел	Создаваемые файлы и артефакты	Ключевой критерий успешной проверки
Раздел 1 (Теория)	Ответ на Вопрос 22 в отчёте	Разграничены переходы **`Running`** → **`Ready`** (ядро) и **`Running`** → **`Waiting`** (процесс), описана роль PCB.
Раздел 2 (Теория)	Ответы на Вопросы 22 и 52	Раскрыты ключи **`rd /S /Q`**, **`taskkill /IM /F`**, коды **`%ERRORLEVEL%`** и фильтрация **`tasklist`**.
Раздел 3 (Файлы)	**`result\actions.txt`**, **`backup\`**, **`evidence\help_copy.txt`**,	Каталог **`discard`** удалён; в **`source`** остались **`copy_a.txt`** и **`note.log`**, а **`move_b.txt`**